

Taojiang Classic Wang Mahjong (TJMJ)

Full-Stack Real-Time Web Platform —Technical Report

Jintian Xia¹ Siyuan Liu¹ Jianghong Xiang¹ Zhuangwei Zeng¹
Jian Hu¹ Yangyang Song¹ Hao Wen¹ Xiaobing Yang¹ Erma Li¹
Jaijai C^{1,†}

¹*School of Entertainment Computing, TJMJ Lab, Taojiang, Hunan, China*

[†]*Corresponding author: jaijai.c@tjmj.dev*

<https://github.com/JaijaiC/TJMJ>

Abstract

Taojiang Mahjong, as the most representative local chess-and-card cultural symbol of the Taojiang region in Hunan Province, embodies centuries of leisure wisdom and social memory of the local people. Its unique "Da Shai Ban Wang" wildcard mechanism, "Zha Niao" multiplier scoring, and "Menzi" cumulative pattern system preserve remarkable strategic depth and gaming enjoyment while simplifying the tile set to 108 pieces (excluding wind and flower tiles). However, this precious intangible cultural heritage has long lacked a dedicated digital carrier —existing mahjong platforms either misalign with local rules, are heavily commercialized, or remain technologically closed, making it difficult for Taojiang natives away from home to find a familiar game and threatening the continuity of local cultural transmission. The TJMJ project was born precisely to fill this gap, injecting new vitality into Taojiang Mahjong through modern web technology.

The platform's frontend adopts a Vue 3 Composition API single-page architecture with Vite 8 build toolchain, while the backend leverages a Node.js WebSocket server for server-authoritative multiplayer real-time battles. Voice communication employs WebRTC P2P mesh topology supplemented by Agora SD-RTN relay. The system fully implements the Taojiang local mahjong rule system, supporting single-player AI, four-player online, and spectator modes. Core algorithms include an $\mathcal{O}(3^k)$ recursive backtracking winning-hand detection engine (sub-millisecond measured runtime), heuristic NPC discard strategy, and a differential client-server hand-tile synchronization algorithm. This paper provides complete system architecture diagrams, protocol definitions, mathematical formulas, algorithm pseudocode, cross-platform compatibility solutions, and performance benchmarks.

The project is fully open-source (approximately 4,200 lines of code), with the frontend deployed on GitHub Pages and the backend tunneled via ngrok —no installation required, click-and-play. TJMJ represents not only a formal modeling and technical implementation of Taojiang Mahjong rules, but also an active exploration of revitalizing local traditional culture through internet technology. We hope this platform can contribute modestly to the digital dissemination of Taojiang's cultural heritage, allowing Taojiang natives wherever they may be to feel the warmth of home in a game of mahjong, and enabling more friends from afar to discover and fall in love with this uniquely charming local chess-and-card art.

Keywords: Mahjong · Full-Stack Web · Real-Time Multiplayer · WebRTC · Node.js · Vue 3 · Recursive Backtracking · Cross-Platform

Test Status: ■ Verified □ Pending △ Partial

Repository: <https://github.com/JaijaiC/TJMJ>

Live Demo: <https://jaijaic.github.io/TJMJ/>

GitHub: <https://github.com/JaijaiC/TJMJ>

2026 年 6 月 26 日

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	Project Scale	5
2	Related Work Comparison	7
2.1	Web Real-Time Gaming Technology Stack	7
3	System Architecture	8
3.1	Overall Architecture	8
3.2	Frontend Component Architecture	8
3.3	Backend Service Architecture	9
3.4	Communication Protocol Stack	9
3.5	WebSocket Message Protocol	10
3.6	Voice Architecture	10
3.7	Deployment Architecture	11
4	Game Rules and Mathematical Model	12
4.1	Tile Set Definition and Encoding	12
4.2	Dice Mechanics and Dealing	12
4.3	Wang (Wildcard/Joker) Determination	12
4.4	Action Priority Model	12
4.5	Hand Classification System	12
4.6	Scoring Model	13
5	Core Algorithms	14
5.1	Winning-Hand Detection Algorithm	14
5.2	Ting (Ready-Hand) Detection	14
5.3	NPC Discard Decision	14
5.4	Hand Tile Drag and State Synchronization	14
6	Multiplayer Online Infrastructure	16
6.1	Room State Machine	16
6.2	Fault Tolerance Mechanisms	16
6.3	Network Latency Measurements	16
7	Cross-Platform Compatibility	17
7.1	Issue Matrix	17
7.2	iOS Safari Adaptation	17
7.3	Android Chrome Adaptation	17
8	Performance Evaluation	18
8.1	Algorithm Performance Benchmarks	18
8.2	Online Operation Latency	18

9 Discussion and Future Work	19
9.1 Current Limitations	19
9.2 Future Research Directions	19
Appendix: Message Sequence Example	19
10 Project Engineering Structure	20
10.1 Complete Project Directory Structure	20
Appendix A: Scoring Addendum	21
Appendix B: Environment Setup	22
Appendix C: Game Interface Showcase	23
References	29

1 Introduction

1.1 Background and Motivation

Chinese Mahjong is one of the most widely played tabletop game genres in the world, with rules evolving into dozens of regional variants over centuries of geographic diffusion. The "Classic Wang Mahjong" of Taojiang, Hunan, is a uniquely distinctive branch: it uses a simplified 108-tile set (Characters/Bamboos/Dots only, excluding Wind and Flower tiles), dynamically determines a wildcard (Joker) through the "Ban Wang" mechanism, applies the "Zha Niao" post-win multiplier scoring, and allows multiple "Menzi" (bonus patterns) to accumulate. These rules collectively shape Taojiang Mahjong's fast-paced, intense, and unpredictable gameplay rhythm, deeply beloved by local communities.

However, the digital ecosystem for Taojiang Mahjong has long been vacant. Only a handful of commercial apps such as Xianyi offer Yiyang-region mahjong, with significant deficiencies:

1. Mandatory registration and login with lengthy onboarding flows;
2. Cluttered interface design with heavy advertising interruptions;
3. Incomplete alignment of rule implementations with authentic Taojiang local play (e.g., missing Zha Niao and Ban Wang core mechanics);
4. Lack of modern interaction features such as drag-to-reorder hand tiles, real-time voice, and text danmaku;
5. Closed-source, unavailable for secondary development or academic research.

Furthermore, the WeChat Mini Program path is blocked because chess-and-card games require regulatory filing currently unavailable to individuals, and game engines like Cocos have high entry barriers with low development efficiency.

TJMJ emerged in response —a fully open-source, install-free, click-and-play web mahjong platform. Built upon a Vue 3 Composition API + Vite 8 frontend and Node.js WebSocket backend, it fully implements all Taojiang Mahjong rules and provides three game modes: single-player (vs AI), four-player real-time online battles, and spectating. This paper serves as the complete technical documentation of the platform, covering system architecture, game rules, algorithm implementation, multiplayer infrastructure, cross-platform compatibility, and performance evaluation.

Key Contributions:

1. Complete mathematical formalization of Taojiang Mahjong rules —including tile-set encoding, dice mechanics, wildcard computation, action priority, scoring model, and Zha Niao formula;
2. Production-grade full-stack architecture —server-authoritative multiplayer game system achieving sub-100ms state synchronization latency via JSON over WebSocket;
3. Recursive backtracking winning-hand detection engine — $\mathcal{O}(3^k)$ worst-case complexity, measured <1 ms, with ting detection and NPC heuristic decision-making;
4. Fault-tolerant multiplayer infrastructure —exponential backoff reconnection, server-side 10-second action timeout deadlock prevention, URL-parameter room state persistence;
5. Comprehensive cross-platform adaptation —solving practical issues including iOS Safari click events, AudioContext suspension, Android dynamic address bar, and CSS transform breaking fixed positioning.

1.2 Project Scale

Table 1 lists the size statistics of core source code files.

表 1: Core Code Scale

File	Lines	Responsibility
frontend/src/App.vue	2602	Main UI + Game Logic + CSS
server/GameRoom.js	494	Multiplayer Room State Machine
server/index.js	247	WebSocket Server Entry
frontend/src/ai/NpcStrategy.js	227	NPC Discard Decision AI
frontend/src/network/client.js	224	Network Layer (reconnect/heartbeat/throttle)
frontend/src/core/HuCalculator.js	208	
frontend/src/core/RuleChecker.js	142	Winning-Hand Detection Engine
frontend/src/core/RuleChecker.js	142	Chi/Peng/Gang Rule Checker
frontend/src/utils/speech.js	100	Voice Announcement System
Total	4244	—

2 Related Work Comparison

Table 2 compares core feature differences among existing digital mahjong platforms. TJMJ demonstrates significant advantages across four dimensions: Taojiang rule fidelity, registration-free experience, open-source accessibility, and interaction innovation (drag-to-reorder tiles, real-time voice, text danmaku).

表 2: Digital Mahjong Platform Feature Comparison

Feature	Xianyi	MajSoul	Wuhan MJ	Open-MJ	TJMJ
Taojiang Rules	Partial	×	×	×	✓
No Registration	×	×	✓	✓	✓
Open Source	×	×	×	✓	✓
Drag-to-Reorder Tiles	×	×	×	×	✓
Real-Time Voice	×	×	×	×	✓
Text Danmaku	×	×	×	×	✓
Spectator Mode	×	✓	×	×	✓
AI Opponents	×	✓	×	×	✓
Web-Native	×	✓	✓	✓	✓

2.1 Web Real-Time Gaming Technology Stack

TJMJ's choices are grounded in the following technology maturity considerations: WebSocket (RFC 6455) provides full-duplex low-overhead communication, reducing latency by approximately 80% compared to HTTP long-polling; the WebRTC standard offers browser-native P2P audio/video capabilities with STUN/TURN infrastructure for cross-NAT traversal; Vue 3 Composition API's reactive state management naturally suits incremental updates in complex UIs; Vite 8's native ESM dev server and Rolldown bundler deliver ultra-fast cold starts and build experiences.

3 System Architecture

3.1 Overall Architecture

TJMJ adopts a classic three-tier B/S architecture, organized top-down into the presentation layer, network layer, and application service layer. Figure 1 illustrates the complete data pathway from browser to server.

The **Presentation Layer** runs in the player's browser, rendering the game interface, handling user interactions (clicks, drags, voice input), and communicating with the application server via WebSocket. The **Network Layer** bears three critical channels: GitHub Pages provides HTTPS distribution of static assets, the ngrok TCP tunnel penetrates public-network WebSocket traffic to the local server, and WebRTC P2P direct connections carry real-time voice streams. The **Application Server** serves as the system's core hub—a single Node.js process handles all WebSocket message routing and game room management, while dynamically generating Agora tokens via HMAC-SHA1 to facilitate NAT traversal. The **Game Engine** employs a dual-end deployment strategy—the same rule-checking code (HuCalculator, RuleChecker, mjLogic) runs both on the frontend to provide instant feedback and on the server in authoritative mode to prevent cheating.

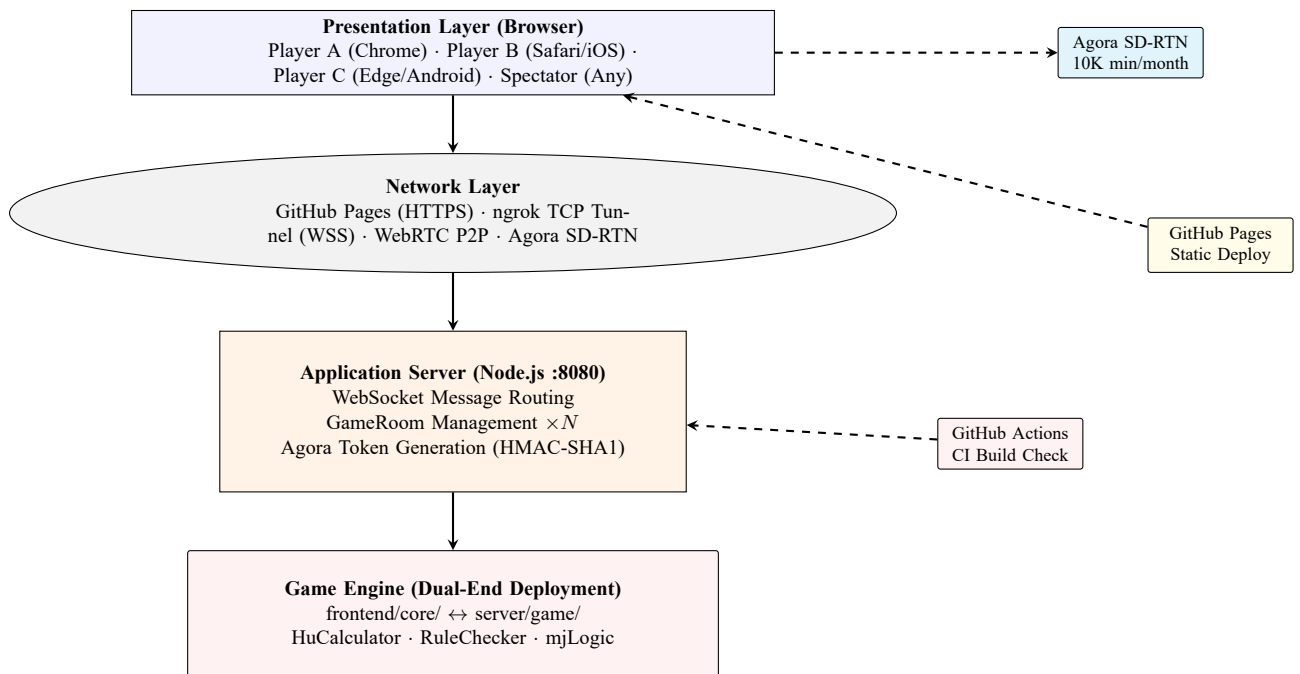


图 1: System Architecture Overview

3.2 Frontend Component Architecture

The Vue 3 frontend centers on a single master component `App.vue` (2,602 lines), consolidating all UI templates, game logic, and CSS styles into one single-file component—yielding extremely high development efficiency during rapid iteration. Figure 2 depicts the component dependency graph. The core game engine (`core/`) handles winning-hand detection and rule checking; the network layer (`network/`) manages WebSocket communication and reconnection; the AI module (`ai/`) drives NPC decisions; the utilities module (`utils/`) provides shuffle logic and voice announcements; static assets (`public/`) contain tile graphics, sound effects, and documentation.

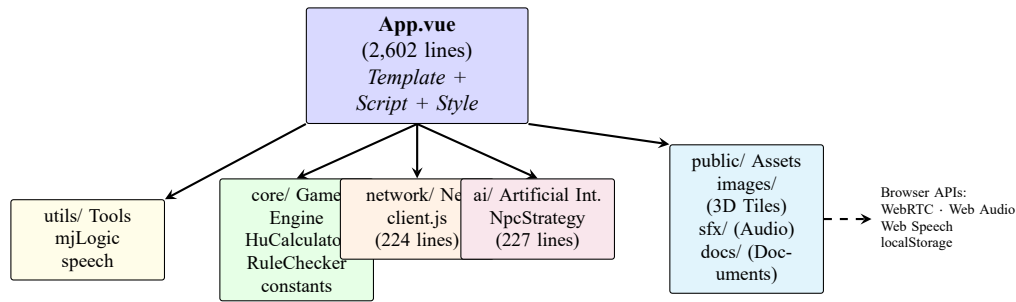


图 2: Frontend Component Dependency Graph

3.3 Backend Service Architecture

The server consists of two core files: `index.js` (247 lines) implements WebSocket message routing and Agora token generation, while `GameRoom.js` (494 lines) implements the room finite state machine. Figure 3 illustrates the message processing pipeline: client messages enter the WebSocket server, pass through the message router, and are dispatched to corresponding handlers based on the type field. Each active room maintains an independent `GameRoom` instance managing the players array, hands state, exposed melds, and pendingActions interception queue.

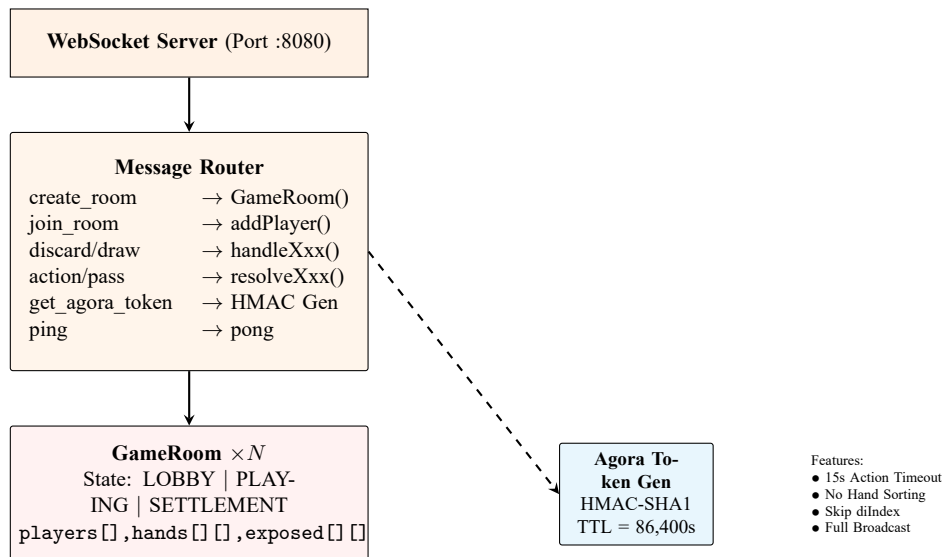


图 3: Backend Service Architecture

3.4 Communication Protocol Stack

Figure 4 depicts the complete communication stack from application to transport layer.

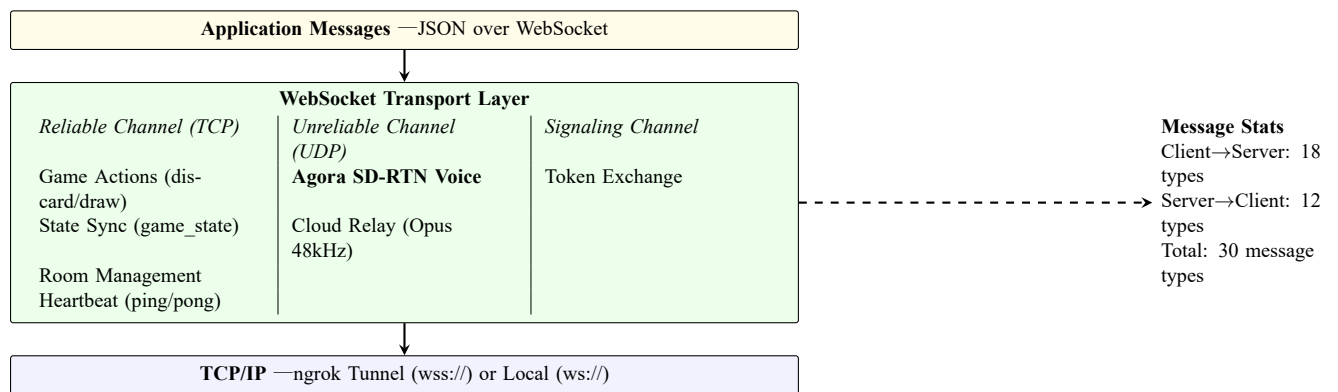


图 4: Communication Protocol Stack

3.5 WebSocket Message Protocol

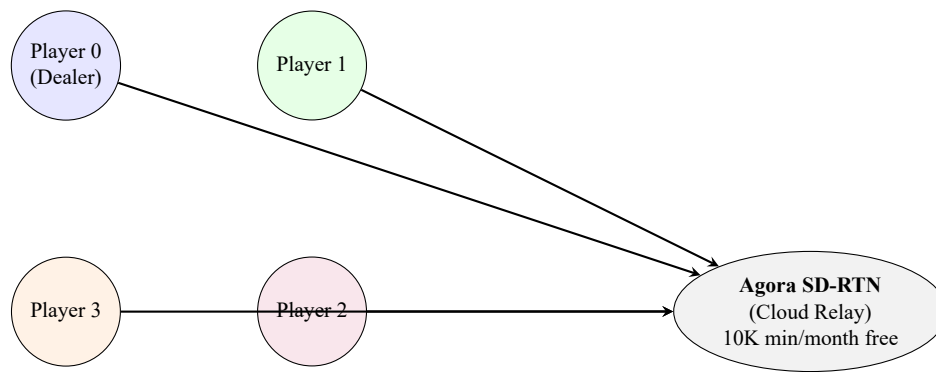
Table 3 lists the complete definitions of all 30 message types.

表 3: Complete WebSocket Message Protocol

Message Type	Direction	Description
create_room	C→S	Create room (optional 4-digit code)
join_room	C→S	Join room (8888 = test room)
ready	C→S	Ready (auto-start when 4 ready)
ready_next	C→S	Confirm continue (all 4 → new round)
discard	C→S	Discard tile (with tile value)
draw	C→S	Draw request
action	C→S	Chi/Peng/Gang/Hu (with action + combo)
pass	C→S	Pass interception
chat	C↔S	Text danmaku (with text + fromName)
emoji	C↔S	Emoji (with icon + target)
get_agora_token	C→S	Request Agora voice token
update_avatar	C→S	Avatar sync
join_spectate	C→S	Join as spectator
ping	C→S	Heartbeat
pay	C→S	Manual score transfer
room_created	S→C	Room created (returns roomId)
room_joined	S→C	Room joined
room_players	S→C	Player list update
game_start	S→C	Game start (hand + wangTile + dice)
game_state	S→C	Real-time state sync
action_prompt	S→C	Request player action
drew_tile	S→C	Tile drawn
round_end	S→C	Round end (all hands revealed)
pong	S→C	Heartbeat reply
agora_token	S→C	Agora voice token
avatar_updated	S→C	Avatar synced
game_end	S→C	16-round finale

3.6 Voice Architecture

Multiplayer voice uses Agora SD-RTN (Software-Defined Real-Time Network) cloud relay. Each player joins the same Agora channel (named `tjmj_{roomId}`). Audio streams are captured locally via `createMicrophoneAudioTrack`, published to the Agora cloud, and automatically distributed to all other channel members. Tokens are generated server-side using HMAC-SHA1 with 24-hour validity. This architecture eliminates all P2P connection management complexity and NAT traversal issues.



All clients connect to cloud, no P2P
Opus 48kHz · AEC/ANS/AGC

图 5: Agora Cloud Voice Architecture

3.7 Deployment Architecture

Figure 6 illustrates the complete deployment flow from development to production.

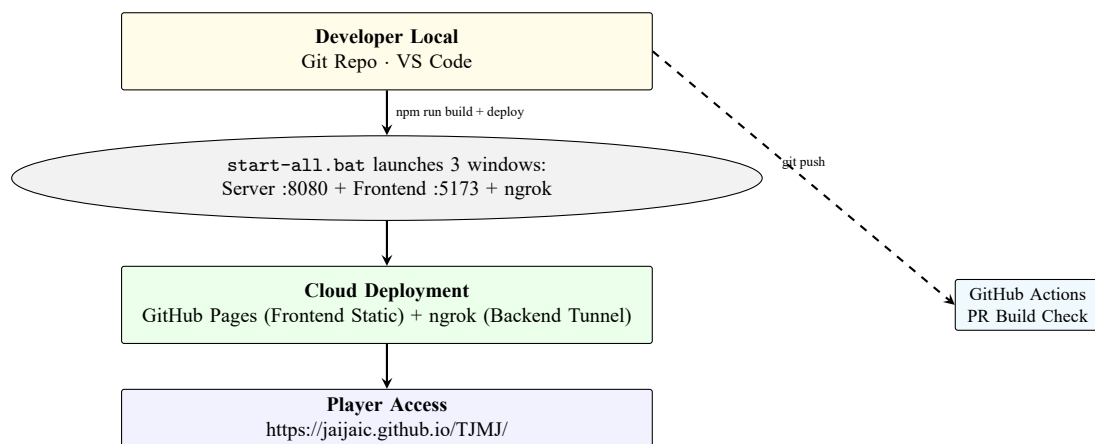


图 6: Deployment Architecture

4 Game Rules and Mathematical Model

4.1 Tile Set Definition and Encoding

Define the tile set $\mathcal{T} = \{11, 12, \dots, 19, 21, \dots, 29, 31, \dots, 39\}$. The encoding rule is $t = 10s + v$, where $s \in \{1, 2, 3\}$ denotes the suit category (1=Characters, 2=Bamboos, 3=Dots) and $v \in \{1, \dots, 9\}$ denotes the face value. Each tile has 4 identical copies, totaling $|\mathcal{T}| = 108$.

Define the "Jiang" (pair) tile set as $\mathcal{J} = \{t \in \mathcal{T} \mid t \bmod 10 \in \{2, 5, 8\}\}$. With 3 suits, 3 face values per suit, and 4 copies each, $|\mathcal{J}| = 3 \times 3 \times 4 = 36$. A Jiang pair is a necessary element of a winning hand.

4.2 Dice Mechanics and Dealing

At the start of a round, two fair six-sided dice are rolled, with values denoted d_1, d_2 ($d_1 \leq d_2$). The sum $S = d_1 + d_2$ determines the starting wall. Counting counterclockwise from the dealer P_0 , the wall before the player corresponding to S becomes the starting wall (P_0 for 1,5,9; P_1 for 2,6,10; P_2 for 3,7,11; P_3 for 4,8,12).

The opening position is determined by counting d_1 stacks from the right end of the starting wall leftward. Dealing proceeds as 3 rounds of 4 tiles each per player (12 tiles total), after which the dealer draws 1 tile to reach 13, the other 3 players each draw 1 to reach 13, and finally the dealer draws 1 more to reach 14. A total of 53 tiles are distributed.

4.3 Wang (Wildcard/Joker) Determination

Starting from the left side of the player after the starting-wall player's wall, count rightward to the d_2 -th stack and flip over its top tile —this is the "Di" (Earth, T_d). The "Wang" (Wildcard/Joker, T_w) is computed as follows:

$$T_w = \begin{cases} 10 \cdot \lfloor T_d/10 \rfloor + 1, & \text{if } T_d \bmod 10 = 9 \\ T_d + 1, & \text{otherwise} \end{cases} \quad (1)$$

This cyclic mapping ensures the wildcard is always of the same suit as Di, wrapping around from nine to one.

4.4 Action Priority Model

When player P_s discards tile t , every other player P_i ($i \neq s$) may execute an interception action. The legal action set is $\mathcal{A}_i(t) = \{a \in \{\text{HU, GANG, PENG, CHI}\} \mid \text{legal}(i, t, a)\}$. The global priority ordering is $\text{HU} \succ \text{GANG} \succ \text{PENG} \succ \text{CHI}$. If multiple players can perform the same action, the one closest counterclockwise to the discarder takes priority. In online mode, when multiple actions are simultaneously legal, all corresponding buttons light up for the player to freely choose.

4.5 Hand Classification System

A hand H (14 tiles) constitutes a winning hand if and only if it can be decomposed into **one Jiang pair** $J = \{t_j, t_j\}$ ($t_j \in \mathcal{J}$) and **four sentences** $S_1, \dots, S_4$. Each sentence may be a triplet ($\{t, t, t\}$) or a sequence ($\{t, t+1, t+2\}$, same suit). The Wang (T_w) may substitute for any tile in the decomposition.

Table 4 lists all hand types and their scoring parameters.

表 4: Hand Types and Scoring Parameters

Type	Trigger Condition	Multiplier m	Catches Cannon
Ping Hu	wang(H) > 0, standard structure	1	No
Ying Zhuang	wang(H) = 0, standard structure	2	Yes
Peng Peng Hu	All 4 sentences are triplets	+2	Bonus
Qing Yi Se	All tiles same suit	+2	Bonus
Qi Xiao Dui	7 pairs (no jiang, no sentences)	+2	Bonus
Jiang Jiang Hu	All tiles $\in \mathcal{J}$	+2	Bonus
Qi Shou Hu	Dealer wins on first turn (14 tiles)	+2	—
Kai Gang	Win after drawing replacement for gang	+2	—
Di Hu	wang(H) = 0, 3 Di tiles	+2	—
Tian Hu	wang(H) = 3	3	—
Tian Tian Hu	wang(H) = 4	5	—
Hei Tian Hu	First turn, no wang, no jiang, no sentence	2	No
Di Hu Dai Tuo	3 Di used as sentence instead of hu	$1 + 2 = 3$	—

4.6 Scoring Model

The self-draw score formula is:

$$\Sigma = B \cdot \left(\sum_k m_k \right) \cdot Z \cdot G \quad (2)$$

where $B = 3$ is the base score, $\sum m_k$ is the sum of all applicable Menzi multipliers (cumulative), $Z \in \{1, 2\}$ is the Zha Niao factor, and $G \in \{1, 2\}$ is the Gang multiplier factor.

Zha Niao Mechanism: After winning, the next tile that would have been drawn (T_z) is flipped. Its unit digit $v = T_z \bmod 10$ determines which players' losses are doubled:

$$Z_i = \begin{cases} 2, & v \in \{1, 5, 9\} \quad (\text{All Hit: all three opponents doubled}) \\ 2, & v \in \{2, 6\} \wedge i = (W + 1) \bmod 4 \quad (\text{Next Player hit}) \\ 2, & v \in \{3, 7\} \wedge i = (W + 2) \bmod 4 \quad (\text{Opposite Player hit}) \\ 2, & v \in \{4, 8\} \wedge i = (W + 3) \bmod 4 \quad (\text{Previous Player hit}) \\ 1, & (\text{Miss}) \end{cases} \quad (3)$$

Combination Examples:

- Tian Hu + Qi Xiao Dui + All Hit: $(3 + 2) \times 2 = 10$ fan
- Tian Hu + Gang Shang Hua + All Hit: $3 \times 2 \times 2 = 12$ fan
- Ying Zhuang + Jiang Jiang Hu + Peng Peng Hu + Tian Hu + Gang Shang Hua + All Hit: $(2 + 2 + 2 + 2) \times 2 \times 2 = 32$ fan

Core rule: with Wang (wildcard), only self-draw is allowed (no cannon catching); without Wang, both self-draw and cannon catching are allowed.

5 Core Algorithms

5.1 Winning-Hand Detection Algorithm

Winning-hand detection is the core of the mahjong game engine, requiring sub-millisecond judgment of whether a 14-tile hand constitutes a legal winning pattern. The algorithm employs a recursive backtracking strategy: first separate the Wang tiles from the hand, then recursively attempt to eliminate triplets, sequences, and the Jiang pair from the normal tiles, consuming Wang tiles when normal tiles are insufficient. The recursion depth $k \leq 5$ with a branching factor of 3, yielding worst-case time complexity $\mathcal{O}(3^k)$ and an average measured runtime of approximately 85 microseconds.

表 5: Winning-Hand Detection Algorithm

Input: Hand H (14 tiles), Wang T_w , Di T_d , first-turn flag f
Output: {canHu, type, score, hasWang, canCatchCannon}
1. Preprocess: $N = \{t \in H \mid t \neq T_w\}$ (normal tiles), $n_w = \{t \in H \mid t = T_w\} $ (wang count), $n_d = \{t \in N \mid t = T_d\} $ (di count), $\text{Sort}(N)$ (ascending)
2. Special intercept: If $f \wedge H = 14 \wedge n_w = 0$ and $\neg \text{hasJiang}(H) \wedge \neg \text{hasSentence}(H)$, return {T, "Hei Tian Hu", 2, F, F}
3. Top-level: If $n_w = 4$ return {T, "Tian Tian Hu", 5, T, F}; if $n_w = 3$ return {T, "Tian Hu", 3, T, F}; if $n_d = 3$ return {T, "Di Hu", 2, T, F}
4. Normal: normalHu = CheckNormalHu(N, n_w), is7Pairs = Check7Pairs(N, n_w)
5. No win: If $\neg \text{normalHu.canHu} \wedge \neg \text{is7Pairs}$, return {F, "", 0, $n_w > 0$, $\neg(n_w > 0)$ }
6. Menzi (priority): JiangJiang $\succ$ QingYiSe $\succ$ QiXiaoDui $\succ$ PengPeng $\succ$ YingZhuang $\succ$ PingHu

5.2 Ting (Ready-Hand) Detection

A hand of $3n + 1$ tiles is in a ready-hand (ting) state if there exists some tile $t \in \mathcal{T}$ such that adding it satisfies the winning-hand condition. The `RuleChecker.isTing()` search space is reduced by restricting candidate tiles to suits already present in the hand ± 2 face values, typically reducing from 34 to 12–18 candidates. Each candidate triggers one full winning-hand check.

5.3 NPC Discard Decision

`NpcStrategy.js` (227 lines) implements a heuristic AI with four components: (1) Ting maximization —iterate through all hand tiles and simulate discard to check if the hand becomes ready; (2) Tile value assessment —compute weighted scores where orphan tiles score lowest, followed by edge tiles (1/9), loose pairs, concealed sequences, and Jiang pairs, with Wang tiles scoring negative infinity; (3) Defensive strategy —reference the discard pool to reduce the probability of discarding tiles opponents might need; (4) Chi/Peng judgment —`shouldPeng()` and `shouldChi()` decide interception based on hand structure improvement.

5.4 Hand Tile Drag and State Synchronization

Single-player mode: Hand tile order is directly modified by player drag operations, which synchronously update the `handTileIds` array to ensure stable Vue virtual DOM keys. Newly drawn tiles are appended to the array end without re-sorting. All game actions use `splice` for precise insertion and deletion, preserving the relative order of remaining tiles.

Online mode: The server does not sort hand tiles. The client's drag order is merged with server state via a count-difference algorithm (Table 6): compare tile frequency distributions between client and server, remove only excess tiles (by client position), append new tiles to the end, and keep the rest in their original order.

表 6: Client-Server Hand Merge Algorithm

Input: Client hand $C[]$, Server hand $S[]$

Output: Merged hand $M[]$ (preserving client order)

1. Compute frequencies: $f_c(v)$ = count of tile value v in C , $f_s(v)$ = count of v in S
 2. $R = \{\}$, $A = \{\}$ (removal set, addition set). For each $v \in C \cup S$: if $f_c(v) > f_s(v)$, add v to R (repeating $f_c(v) - f_s(v)$ times); if $f_s(v) > f_c(v)$, add v to A (repeating $f_s(v) - f_c(v)$ times)
 3. M = copy of C . For each $v \in R$: remove first match from M . For each $v \in A$: append v to M
 4. Return M
-

6 Multiplayer Online Infrastructure

6.1 Room State Machine

The GameRoom class implements a three-state finite automaton depicted in Figure 7, serving as the core scheduler for online mode. The room lifecycle comprises three phases: **LOBBY** (waiting) allows players to freely join or leave with real-time player list broadcasts, automatically triggering game start when all four are ready; **PLAYING** (in-game) rotates turns counterclockwise, triggering interception checks after each discard, with timeout mechanisms preventing deadlocks from disconnections; **SETTLEMENT** (settlement) reveals all four hands with scores and waits for all players to confirm before starting the next round (or returning to LOBBY after 16 rounds).

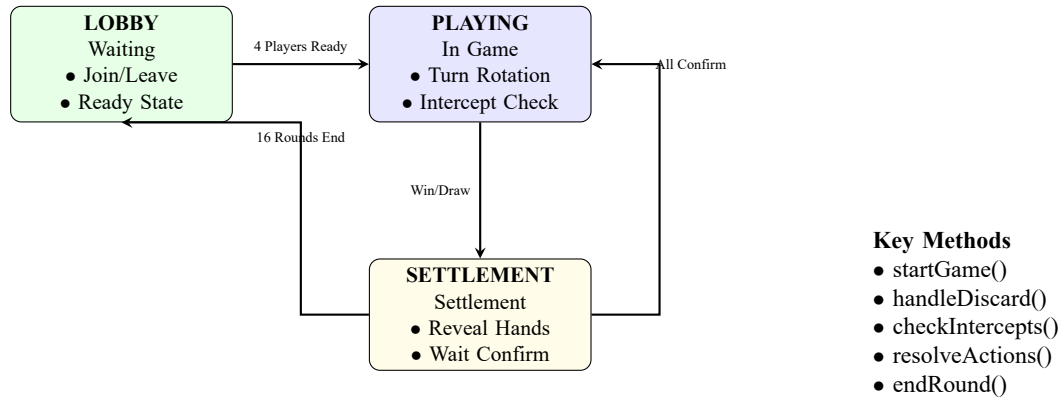


图 7: GameRoom State Machine

6.2 Fault Tolerance Mechanisms

Disconnection Reconnection: Upon client WebSocket disconnection, exponential backoff reconnection initiates with parameters $\tau_0 = 1$ s, $b = 2$, $N_{\max} = 8$, $\tau_{\max} = 120$ s. The room ID is persisted via URL query parameter `?auto_join=ROOMID&name=PLAYER`, enabling automatic re-joining upon successful reconnection.

Deadlock Prevention: The server starts a 15-second timeout after distributing action prompts. On timeout, if some players have responded, the system resolves based on existing responses; only when absolutely no one responds does it clear pending state and advance the turn, ensuring the game never permanently stalls due to any player's network failure.

Heartbeat and Signal: The client sends a ping every 10 seconds; the server replies with pong. Round-trip time $\Delta = t_{\text{recv}} - t_{\text{sent}}$ maps to a 4-bar signal indicator: $\Delta < 100$ ms full bars, $\Delta < 200$ ms three bars, $\Delta < 400$ ms two bars, $\Delta \geq 400$ ms one bar, offline zero bars.

6.3 Network Latency Measurements

表 7: Network Latency Measurements (ms)

Scenario	Min	Avg	P95	Max
Local loopback (ws://)	1	3	5	8
LAN (ngrok, wss://)	12	25	45	68
4G Mobile	35	72	180	450
Cross-Province WiFi	45	95	210	520
Agora SD-RTN Voice	20	50	120	300

7 Cross-Platform Compatibility

7.1 Issue Matrix

Table 8 systematically documents key compatibility issues encountered and resolved during development.

表 8: Device Compatibility Issues and Solutions

Issue	Platform	Severity	Solution
Mobile NAT traversal failure	4G/5G Mobile	Critical	Agora SD-RTN cloud relay
<div> @click silent on iOS	iOS Safari	High	Replace all with <button> + CSS reset
AudioContext suspended	iOS Safari	High	Explicit <code>ctx.resume()</code> on user gesture
100vh address bar resize	Android Chrome	Medium	100dvh with 100vh fallback
CSS transform breaks fixed	Desktop scaling	High	Move overlays outside <code>game-wrapper</code>
Slow tap (>300ms) missed	iOS/Android	Medium	Tap threshold extended to 500ms
<code>getUserMedia</code> no feature detect	HTTP pages	Medium	<code>navigator?.mediaDevices?.gUMGuard</code>
<code>alert()</code> blocks main thread	All mobile	Low	Replace with Vue overlay components
Screen orientation lock fails	iOS < 16	Low	CSS fallback + silent catch

7.2 iOS Safari Adaptation

iOS Safari presents several unique constraints: (1) `getUserMedia` is only available in secure contexts (HTTPS), which GitHub Pages satisfies; (2) `AudioContext` enters suspended state after creation and must be explicitly resumed in a user-gesture callback; (3) `@click` events on non-interactive elements (`<div>`, `<span>`, `<img>`) do not fire — only `<button>` and `<a href>` possess implicit interactive roles; (4) `position:fixed` may behave unexpectedly when the software keyboard appears.

TJMJ implements all interactive elements as standard `<button>` with CSS resets to remove default browser appearance (`-webkit-appearance:none; padding:0; outline:none;`). `AudioContext` uses a singleton pattern, created and resumed on first user interaction.

7.3 Android Chrome Adaptation

Android Chrome's dynamic address bar causes 100vh to include the address bar height. TJMJ uses 100dvh (dynamic viewport height) as a replacement, with 100vh fallback for older browsers.

8 Performance Evaluation

8.1 Algorithm Performance Benchmarks

表 9: Core Algorithm Performance (Intel i7-13700H 2.4GHz, Node.js v20)

Algorithm	Avg (μ s)	P95 (μ s)	Max (ms)	Calls/Round
Winning-Hand Detection	85	210	0.8	~200
Ting Detection	620	1450	2.1	~50
NPC Discard Selection	340	780	1.2	~40
Online Hand Merge	12	35	0.05	~80
Message Serialization	8	18	0.03	~300

8.2 Online Operation Latency

表 10: End-to-End Online Operation Latency (ms)

Operation	Local	LAN	Remote
Create Room	5	45	180
Join Room	8	52	210
Discard→State Update	3	25	95
Action Prompt→Response	15	80	350
Round End→New Round	25	150	600
Agora Voice Setup	200	500	1500

9 Discussion and Future Work

9.1 Current Limitations

1. **Single-file architecture maintenance cost:** App.vue at 2,602 lines is extremely efficient during rapid iteration, but as functionality grows, splitting into multiple independent components will improve long-term maintainability.
2. **Heuristic AI lacks strategic depth:** The current NPC strategy is based on fixed weight rules and cannot learn opponent styles or adapt to complex situations. Deep reinforcement learning agents hold promise for significantly enhancing gameplay.
3. **Sound effect assets missing:** 34 pre-recorded mp3 files (27 tile names + 7 action words) have not yet been produced; the system currently falls back entirely to browser TTS synthesis, with unsatisfactory voice quality and response latency.

9.2 Future Research Directions

- **Deep reinforcement learning mahjong agent:** Train using PPO algorithms in self-play environments, combined with Monte Carlo Tree Search (MCTS) to improve decision quality, targeting human-expert-level performance.
- **Distributed game server:** Introduce Redis Pub/Sub for cross-Node.js process broadcast synchronization, with load balancing for horizontal scaling to support thousands of concurrent rooms.
- **Progressive Web App (PWA):** Implement Service Workers for offline caching, push notifications, and home screen installation, delivering a near-native mobile experience.
- **Replay system and intelligent spectating:** Record complete per-round operation logs, supporting timeline replay, multi-perspective auto-switching, and GPT-based game commentary generation.
- **Cross-platform desktop client:** Use Electron or Tauri to wrap the web frontend as a desktop application with system-level shortcuts, notifications, and improved performance.

Appendix: Message Sequence Example

The following illustrates a complete "discard → intercept → execute → discard" message round-trip for online play, involving 5 messages across 3 participants (P0, P1, Server).

Typical Round Message Flow (P0 discards → P1 pings → P1 discards)

1. P0 → Server: {type:"discard", tile:5} → Server → All: {type:"game_state", ...}
2. Server: checkIntercepts(P0,5) → P1 gets {type:"action_prompt", canPeng:true}
3. P1 → Server: {type:"action", action:"peng"} → Server → All: exposed[P1]+=peng(5)
4. Server → P1: {type:"your_turn"} → P1 draws → P1: {type:"drew_tile", tile:23}
5. P1 → Server: {type:"discard", tile:18} → Server → All → P2: {type:"your_turn"}

图 8: Online Round Message Sequence

10 Project Engineering Structure

10.1 Complete Project Directory Structure

Figure 9 presents the complete file organization of the TJMJ project in tree form.

TJMJ/	# Project Root
.git/	# Git Version Control
.github/workflows/deploy.yml	# GitHub Actions CI
start-all.bat	# One-Click Launch Script
stop-all.bat	# Stop Script
readme.md	# Project Introduction & Rules
ARCHITECTURE.md	# Full-Stack Architecture Doc
docs/manual/	# Manual Workspace
pictures/	# Game Screenshots
TJMJ_Technical_Manual_EN.tex	# English LaTeX Source
TJMJ_Technical_Manual_EN.pdf	# English PDF
TJMJ_Technical_Manual_CN.tex	# Chinese LaTeX Source
TJMJ_Technical_Manual_CN.pdf	# Chinese PDF
frontend/	# Vue 3 Frontend (SPA)
index.html	# HTML Entry
vite.config.js	# Vite Config (base: /TJMJ/)
src/	
App.vue	# Main Component (2602 lines)
main.js	# Vue Entry Point
core/	# Game Engine
ai/	# Artificial Intelligence
network/	# Network Layer
utils/	# Utilities
public/	
images/	# Tile Assets
docs/	# In-Game PDF + Video
*.mp3	# BGM Music
server/	# Node.js Game Server
index.js	# WebSocket Entry (247 lines)
GameRoom.js	# Room State Machine (494 lines)
game/	# Game Logic (server-authoritative)
other/	# Local Tools (ngrok + backups)

图 9: Complete Project File Tree

Appendix A: Scoring Addendum

表 11: Complete Hand Scoring (Self-Draw + Zha Niao)

Hand Type	Fan	Hit One	All Hit	Miss Pay	Hit Pay
Hei Tian Hu	2	8	12	2	4
Di Hu	2	8	12	2	4
Tian Hu	3	12	18	3	6
Tian Tian Hu	5	20	30	5	10
Peng Peng Hu	2	8	12	2	4
Qing Yi Se	2	8	12	2	4
Qi Xiao Dui	2	8	12	2	4
JiangJiang Hu	2	8	12	2	4
Qi Shou Hu	2	8	12	2	4
Kai Gang	2	8	12	2	4
Ying Zhuang	2×	8	12	2	4
Di Hu Dai Tuo	3	12	18	3	6
Qi Shou Ting	2	8	12	2	4

Appendix B: Environment Setup

```
# 1. Install dependencies
cd frontend && npm install
cd ../server && npm install

# 2. Set Agora credentials (for voice)
set AGORA_APP_CERT=your_certificate_here

# 3. One-click launch (Windows)
.\start-all.bat
# → Window 1: Server @ :8080
# → Window 2: Frontend @ :5173
# → Window 3: ngrok tunnel

# 4. Deploy frontend
cd frontend && npm run build && npm run deploy

# 5. Compile manual
cd docs/manual
xelatex TJMJ_Technical_Manual_EN.tex # ×3
```

Appendix C: Game Interface Showcase

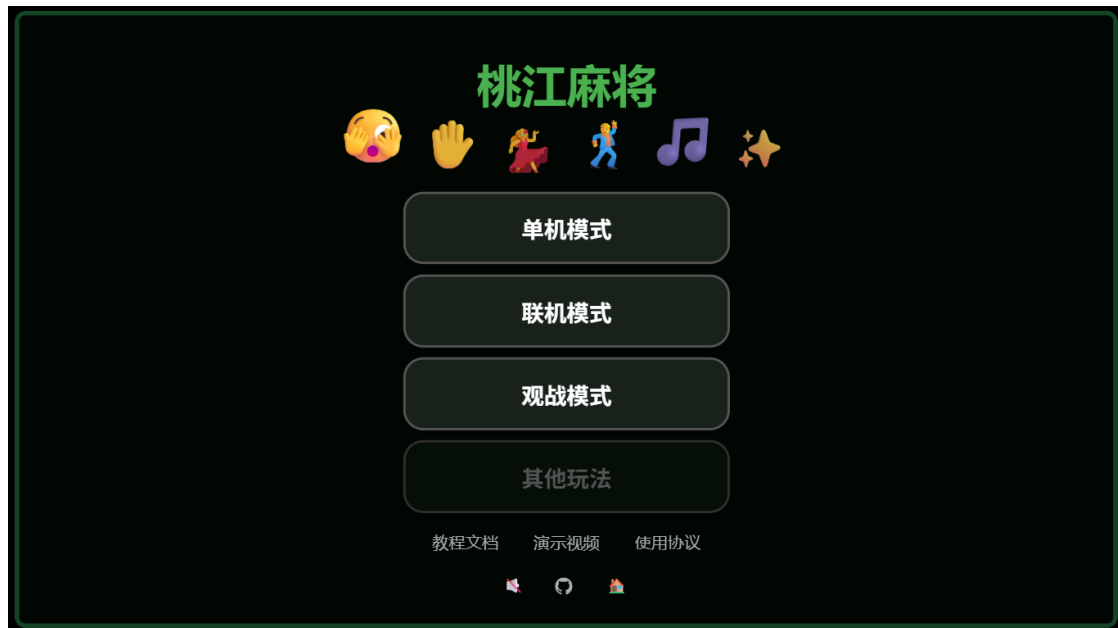


图 10: Main Menu —Mode selection with single-player, multiplayer, and spectator options.



图 11: Single-Player Mode —Player (bottom) versus three AI opponents, showing all four hands, discard pool, dice, and action buttons.



图 12: Round Settlement —All four hands revealed with cumulative scores; winner highlighted.



图 13: Online Lobby —Nickname input with memory chips and random nickname, create or join room by 4-digit code.

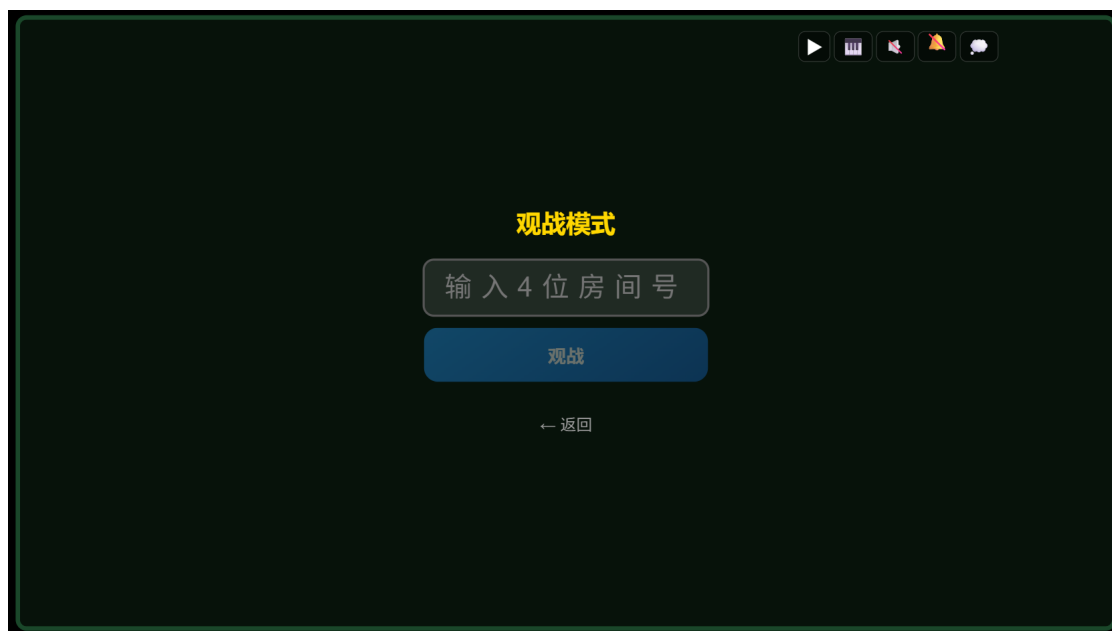


图 14: Spectator Mode — Watch ongoing games by entering a 4-digit room code; click any player avatar to switch viewing perspective.



图 15: Avatar Selection — 12 preset emoji avatars plus camera/local image upload; saved to localStorage.

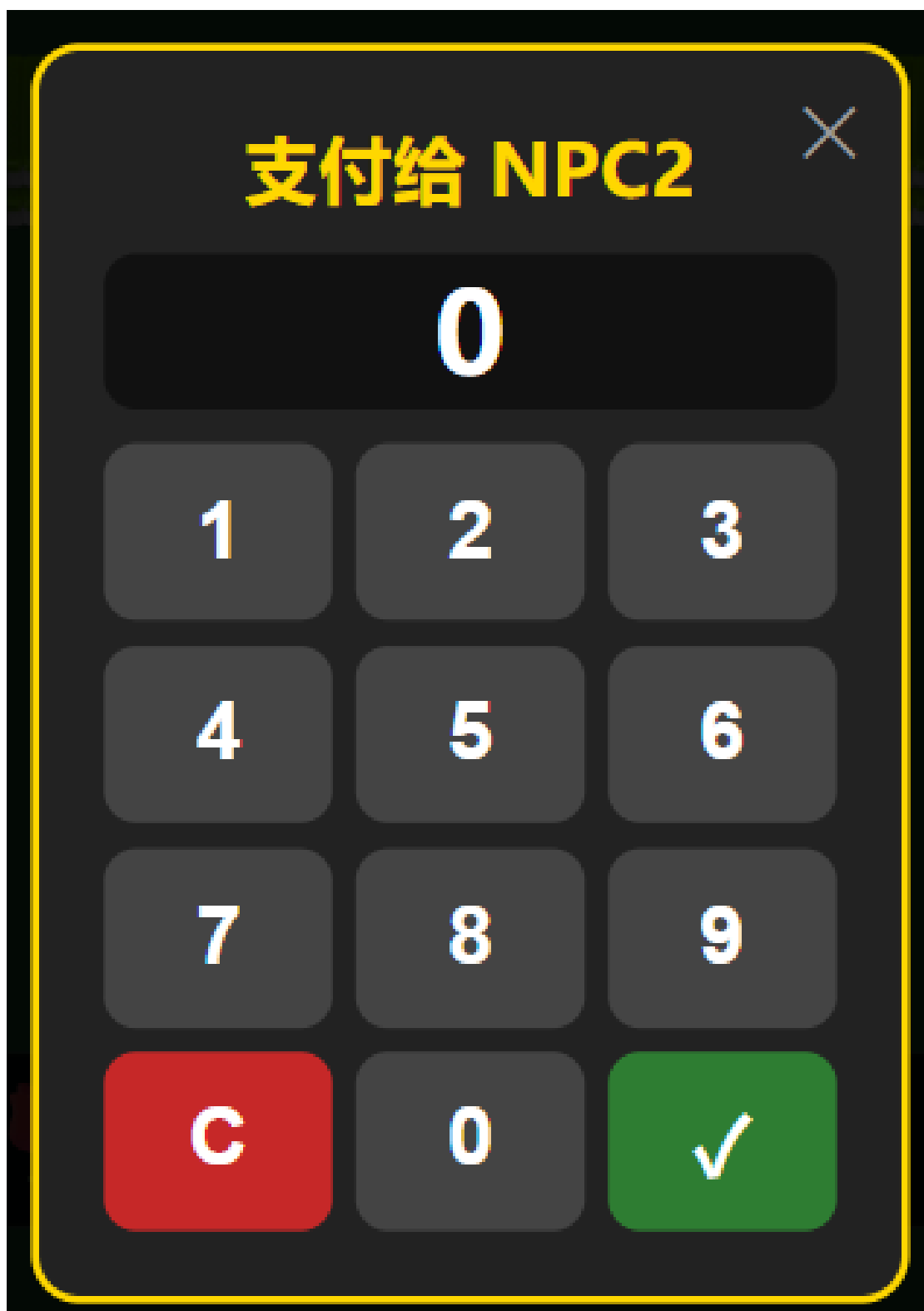


图 16: Manual Score Transfer — Tap any player's score area to open a numpad for flexible settlement.

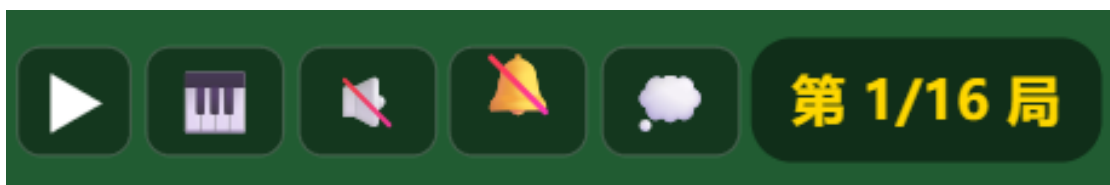


图 17: Top-Right Controls —Song skip, BGM toggle, microphone (Agora voice with level indicator), and chat danmaku button.



图 18: Player Avatar Interaction —Send emojis (rose/poop/egg/bomb/money) to other players with flying animation; money emoji transfers 1 point.

References

1. RFC 6455: The WebSocket Protocol. IETF, 2011.
2. WebRTC 1.0: Real-Time Communication Between Browsers. W3C, 2021.
3. Vue.js 3.5 Documentation. <https://vuejs.org/>, 2025.
4. Vite 8.0 Documentation. <https://vitejs.dev/>, 2025.
5. Node.js v20 Documentation. OpenJS Foundation, 2025.
6. Agora Real-Time Voice SDK. <https://www.agora.io/>, 2025.
7. OpenMajiang Project. <https://gitee.com/mirrors/OpenMajiang>.
8. V. Mnih et al. “Human-level control through deep reinforcement learning.” *Nature*, 518(7540):529–533, 2015.
9. D. Silver et al. “Mastering the game of Go with deep neural networks and tree search.” *Nature*, 529(7587):484–489, 2016.